

Introduction to Data Analysis

Week 2. Data Types and Data Structures

Jin Hee Yoon

2026

Sejong University

Week 2 Overview

2주차에서는 **Data Types**와 **Data Structures**를 학습한다. 첫 번째 영상에서는 데이터와 변수의 유형, structured data의 구조를 이론적으로 이해하고, 두 번째 영상에서는 Python의 기본 자료구조와 pandas DataFrame을 직접 다룬다.

- **Week 2-1:** Data type, numerical/categorical/ordinal data, structured/unstructured data, observation과 variable
- **Week 2-2:** List, Dictionary, NumPy Array, DataFrame, selection, filtering, sorting, grouping, wide/long data

이번 주의 목표는 데이터의 “값”만 보는 것이 아니라, 각 **variable**의 의미와 데이터가 어떤 구조로 저장되어 있는지를 정확히 이해하는 것이다.

Learning Objectives

1. **Numerical, Categorical, Ordinal** data를 구분할 수 있다.
2. **Discrete**와 **Continuous** numerical variable을 설명할 수 있다.
3. **Structured data**와 **Unstructured data**의 차이를 설명할 수 있다.
4. table data에서 **row, column, observation, variable, value**를 구분할 수 있다.
5. Python의 **List, Dictionary, NumPy Array, DataFrame**을 사용할 수 있다.
6. pandas에서 column 선택, filtering, sorting, grouping을 수행할 수 있다.
7. **Wide data**와 **Long data**를 이해하고 변환할 수 있다.

Week 2-1. Data Types and Table Structure

Week 2-1 Overview

- Data와 variable의 개념
- Numerical data와 Categorical data
- Discrete와 Continuous numerical variable
- Nominal과 Ordinal categorical variable
- Identifier와 code를 해석할 때 주의할 점
- Structured, Semi-structured, Unstructured data
- Row, column, observation, variable, value
- Dataset을 설계할 때 고려할 사항

Week 2-1의 목표는 Python 코드를 쓰기 전에 “이 column은 어떤 type이고, 한 row는 무엇을 의미하는가?”를 정확히 설명할 수 있게 되는 것이다.

What Is Data?

Data는 관찰, 측정, 기록을 통해 얻은 값이나 정보의 집합이다.

예를 들어 한 학생에 대해 다음 정보를 기록할 수 있다.

- Student ID: 2026001
- Major: AI
- StudyHours: 3.5
- Attendance: 90
- Final: 82

Data는 숫자만을 의미하지 않는다. 문자, category, 날짜, image, audio, text도 모두 data가 될 수 있다.

Variable and Value

Variable은 observation에 대해 측정하거나 기록하는 특성이다. **Value**는 특정 observation에서 실제로 관측된 값이다.

Variable	의미	Value 예시
Major	전공	AI
StudyHours	공부 시간	3.5
Attendance	출석률	90
Final	기말고사 점수	82

Variable은 “무엇을 측정했는가”이고, Value는 “그 variable에서 실제로 얼마가 관측되었는가”이다.

Data Types

Data variable은 여러 유형으로 분류할 수 있다.

Type	Meaning	Example
Numerical	숫자로 측정되는 variable	height, score, income
Categorical	집단 또는 범주를 나타냄	region, major, blood type
Ordinal	순서가 있는 category	satisfaction level
Time series	시간 순서에 따라 관측	daily sales, temperature
Text	language data	review, comment
Image/audio	multimedia data	photo, speech signal

Variable의 종류에 따라 가능한 summary, visualization, statistical method가 달라진다.

Numerical Data

Numerical data는 양적인 크기나 수량을 숫자로 나타낸다.

대표적인 예:

- 학생의 시험 점수
- 하루 공부 시간
- 제품 판매량
- 고객의 구매 금액
- 센서로 측정한 temperature

숫자로 저장되어 있다는 이유만으로 numerical data라고 판단하면 안 된다. 숫자가 단순한 label이나 code일 수도 있다.

Discrete Numerical Variables

Discrete numerical variable은 보통 셀 수 있는 값을 나타낸다.

예:

- 하루 방문 고객 수: 120명
- 결석 횟수: 3회
- 구매한 제품 수: 5개
- 불량 제품 수: 2개
- 댓글 수: 18개

Discrete variable은 값 사이에 가능한 값이 제한되어 있는 경우가 많다. 예를 들어 고객 수가 12.4명일 수는 없다.

Continuous Numerical Variables

Continuous numerical variable은 측정을 통해 얻으며, 일정 범위 안에서 연속적인 값을 가질 수 있다.

예:

- height: 172.4 cm
- weight: 68.7 kg
- response time: 1.37 sec
- temperature: 24.8 °C
- distance: 3.42 km

실제 데이터 파일에서는 소수점 자리수 때문에 값이 이산적으로 보일 수 있지만, 측정 대상 자체는 continuous variable일 수 있다.

Discrete vs. Continuous

구분	Discrete	Continuous
의미	셀 수 있는 값	측정되는 값
가능한 값	분리된 값	연속적인 값
예	학생 수, 결석 횟수	키, 몸무게, 시간
대표 질문	“몇 개인가?”	“얼마나 큰가?”

시험 점수처럼 정수로 기록되는 값은 분석 목적에 따라 **numerical variable**로 다루지만, 엄밀한 측정 척도 관점에서는 데이터 생성 방식도 함께 고려해야 한다.

Categorical Data

Categorical data는 observation이 어떤 group 또는 category에 속하는지를 나타낸다.

예:

- city: Seoul, Busan, Jeju
- major: AI, Data, Business
- blood type: A, B, AB, O
- product category: Food, Electronics, Clothing
- class label: Normal, Abnormal

Categorical variable은 값의 “크기”보다 “어느 category에 속하는가”가 중요하다.

Nominal Data

Nominal data는 category 사이에 의미 있는 순서가 없는 categorical data이다.

예:

- blood type: A, B, AB, O
- city: Seoul, Busan, Jeju
- major: AI, Data, Business
- product category: Food, Clothing, Electronics

Nominal data에서는 “AI가 Data보다 크다”와 같은 비교는 의미가 없다.

Ordinal Data

Ordinal data는 의미 있는 순서를 가진 categorical data이다.

예:

- satisfaction: very low, low, neutral, high, very high
- risk level: low, medium, high
- education level
- survey response: strongly disagree to strongly agree

Ordinal data에는 순서가 있지만, 인접한 category 사이의 간격이 같다고 볼 수는 없다.

Numbers Can Be Categories

숫자로 저장되어 있어도 numerical variable이 아닐 수 있다.

예를 들어 지역을 다음과 같이 code로 저장했다고 하자.

RegionCode	Region
1	Seoul
2	Busan
3	Jeju

RegionCode 3이 RegionCode 1보다 “3배 크다”고 해석할 수 없다. 숫자는 단지 category를 구분하기 위한 label이다.

Identifier Is Not a Measurement

Student ID, Customer ID, Product ID와 같은 **Identifier**는 observation을 구분하기 위한 값이다.

- ID = 1001, 1002, 1003처럼 숫자로 저장될 수 있다.
- 하지만 ID의 평균이나 표준편차를 계산하는 것은 일반적으로 의미가 없다.
- ID는 numerical variable이 아니라 observation을 식별하기 위한 label로 이해한다.

Data type은 컴퓨터에 저장된 형식만 보는 것이 아니라, variable이 실제로 무엇을 의미하는 지까지 함께 판단해야 한다.

Time Series Data

Time series data는 시간 순서에 따라 반복적으로 관측된 데이터이다.

예:

- 날짜별 매출
- 시간대별 전력 사용량
- 매일 측정한 temperature
- 월별 실업률
- 분 단위 stock price

Time series에서는 값뿐 아니라 관측된 순서가 중요하다. 시간 순서를 섞으면 trend나 seasonality를 해석하기 어렵다.

Text, Image, and Audio Data

Text, image, audio는 전통적인 row-column table과 다른 형태의 data이다.

- **Text:** review, news article, social media comment
- **Image:** X-ray, product photo, satellite image
- **Audio:** speech, music, machine sound

이러한 data도 분석을 위해 숫자 형태의 feature로 변환될 수 있다. 다만 초반 강의에서는 table 형태의 structured data를 중심으로 다룬다.

Structured Data

Structured data는 미리 정해진 schema에 따라 row와 column의 형태로 정리된 data이다.

대표적인 예:

- Excel spreadsheet
- CSV file
- database table
- 설문조사 결과 table
- 학생 성적표

Structured data에서는 각 column이 어떤 variable을 의미하는지 명확하고, 각 row가 같은 구조를 갖는다.

Unstructured Data

Unstructured data는 자연스럽게 일정한 row-column schema를 따르지 않는 data이다.

예:

- text document
- image
- audio file
- video
- free-form email

Unstructured data도 분석할 수 있지만, 보통 분석 전에 text vector, image feature, audio feature 등으로 변환하는 과정이 필요하다.

Semi-structured Data

Semi-structured data는 완전한 table은 아니지만, tag나 key를 통해 어느 정도 구조를 갖는 data이다.

대표적인 예:

- JSON
- XML
- web log
- 일부 API response

Semi-structured data는 구조가 존재하지만 모든 record가 동일한 column을 반드시 갖는 것은 아니다.

Structured vs. Unstructured Data

구분	Structured	Unstructured
형태	row-column schema	자유로운 형태
예	CSV, Excel, DB table	text, image, audio
분석 준비	비교적 바로 가능	feature extraction이 필요한 경우가 많음
초반 강의	중심적으로 사용	개념 소개 중심

이 강의에서 pandas DataFrame으로 다루는 대부분의 데이터는 structured data이다.

Rows and Columns

Table data에서 row와 column의 의미는 매우 중요하다.

- 각 **row**는 보통 하나의 observation을 나타낸다.
- 각 **column**은 보통 하나의 variable을 나타낸다.
- 각 **cell**에는 하나의 value가 들어간다.

Observation	Variable 1	Variable 2	Variable 3
1	value	value	value
2	value	value	value
3	value	value	value

Observation Unit Matters

같은 주제라도 분석 단위에 따라 row의 의미가 달라질 수 있다.

예: 대학교 수업 데이터

- row = 학생 한 명: 학생별 성적 분석
- row = 강의 한 개: 강의별 수강 인원 분석
- row = 시험 문항 한 개: 문항별 정답률 분석
- row = 학생-문항 조합: 세밀한 응답 패턴 분석

분석을 시작하기 전에 “한 row가 무엇을 의미하는가?”를 반드시 확인해야 한다.

Example Dataset: Student Learning

ID	Major	GradeLevel	StudyTime	Final
1	AI	1	2.5	78
2	Data	2	3.0	85
3	AI	3	4.0	92
4	Business	2	1.5	70
5	Data	4	5.0	95
6	AI	1	2.0	80

- 한 row는 한 학생을 의미한다.
- Major는 categorical variable이다.
- StudyTime과 Final은 numerical variable이다.
- GradeLevel은 숫자로 저장되지만 순서 의미를 가진 ordinal variable로 볼 수 있다.

Choosing an Appropriate Data Type

Variable의 type을 잘못 이해하면 분석 방법도 잘못 선택될 수 있다.

Variable	잘못된 해석	적절한 해석
Student ID	numerical	identifier
Major code	numerical	categorical
Satisfaction 1-5	continuous	ordinal
StudyHours	categorical	numerical
Final score	text	numerical

분석에서 중요한 것은 storage type보다 semantic type, 즉 데이터가 실제로 무엇을 의미하는가이다.

From Data Type to Analysis Method

Data type은 이후 분석 방법과 visualization 선택에 직접 연결된다.

Data type	대표 summary	대표 visualization
Numerical	mean, median, std	histogram, boxplot
Categorical	count, proportion	bar plot
Ordinal	frequency, median	ordered bar plot
Two numerical	correlation	scatter plot
Category + numerical	group mean	grouped boxplot, bar plot

Week 2의 data type 이해가 이후 EDA와 visualization 수업의 기초가 된다.

Common Mistakes When Reading a Dataset

초보자가 dataset을 볼 때 자주 하는 실수는 다음과 같다.

- 숫자로 보이면 모두 numerical variable이라고 생각한다.
- row가 무엇을 의미하는지 확인하지 않는다.
- unit을 확인하지 않고 값의 크기를 비교한다.
- missing value를 0과 동일하게 생각한다.
- category code를 실제 크기 순서로 해석한다.
- identifier를 분석 variable로 사용한다.

Dataset을 처음 받았을 때는 계산보다 먼저 variable definition과 data dictionary를 확인하는 습관이 중요하다.

Week 2-1 Summary

Week 2-1에서 배운 내용을 정리하면 다음과 같다.

- Numerical data는 discrete와 continuous로 나눌 수 있다.
- Categorical data는 nominal과 ordinal로 구분할 수 있다.
- 숫자로 저장되어 있어도 ID나 category code는 numerical measurement가 아닐 수 있다.
- Structured data는 row-column schema가 명확한 data이다.
- Table data에서는 row = observation, column = variable, cell = value로 이해한다.
- Data type과 observation unit을 정확히 이해해야 적절한 분석 방법을 선택할 수 있다.

다음 영상에서는 Python에서 이러한 개념이 List, Dictionary, NumPy Array, pandas DataFrame으로 어떻게 구현되는지 직접 실습한다.

Week 2-2. Python Data Structures and Practice

Week 2-2 Overview

Week 2-2에서는 Python으로 data structure를 직접 만들고 조작한다.

- Python basic data structures: List, Dictionary, NumPy Array, DataFrame
- List indexing과 Dictionary의 Key-Value structure
- NumPy의 vectorized computation
- DataFrame, Column, Series, Index의 관계
- Column selection과 Boolean filtering
- Sorting과 Group Summary
- Wide data와 Long data
- Practice examples

코드를 외우는 것보다 “어떤 object에 어떤 method를 적용하고 있는가”를 이해하는 것이 더 중요하다.

Python Data Structures

Python은 여러 가지 기본 data structure를 제공한다.

Structure	특징	Example
List	순서가 있고 수정 가능	[10, 20, 30]
Tuple	순서는 있지만 수정 불가능	(10, 20, 30)
Dictionary	Key-Value pair	{"name": "Kim"}
Set	중복 없는 unique value	{1, 2, 3}
NumPy Array	numerical vector/matrix	np.array([1,2,3])
DataFrame	table data	pd.DataFrame()

이번 실습에서는 데이터분석에서 특히 자주 사용하는 List, Dictionary, NumPy Array, DataFrame을 중심으로 다룬다.

Python Example: List and Indexing

```
# Create a list of final scores
scores = [82, 60, 91, 73, 96]

# Access values by position
print(scores[0]) # first value
print(scores[-1]) # last value

# Check length and calculate an average
print(len(scores))
print(sum(scores) / len(scores))

# Add a new value
scores.append(88)
print(scores)
```

Python의 index는 0부터 시작한다. 따라서 `scores[0]`은 첫 번째 value를 의미한다.

Python Example: Dictionary

```
# Create a dictionary using key-value pairs
student = {
    "ID": 1,
    "StudyHours": 3.5,
    "Attendance": 90,
    "Final": 82 }

# Access values using keys
print(student["ID"])
print(student["Final"])

# Add a new key-value pair
student["Grade"] = "B"
print(student)
```

Dictionary는 **Key-Value Pair**로 데이터를 저장한다. 예를 들어 “Final”은 Key이고 82는 그 Key에 대응하는 Value이다.

Dictionary: Key and Value

Dictionary 구조를 표로 보면 다음과 같다.

Key	Value
"ID"	1
"StudyHours"	3.5
"Attendance"	90
"Final"	82

Dictionary를 DataFrame으로 변환할 때 Dictionary의 Key가 DataFrame의 column name 이 되는 경우가 많다.

Python Example: NumPy Array

```
# Import NumPy with the conventional alias np
import numpy as np

# Convert a list of scores to a NumPy array
x = np.array([82, 60, 91, 73, 96])

# Calculate basic numerical summaries
print(x.mean())
print(x.std())
print(x.min())
print(x.max())

# Apply one formula to all elements at once
z = (x - x.mean()) / x.std()
print(z)
```

Vectorized Computation

Vectorized computation은 많은 값에 같은 연산을 한 번에 적용하는 것을 의미한다.

$$x = (x_1, x_2, \dots, x_n)$$

standardization은 다음과 같이 표현할 수 있다.

$$z_i = \frac{x_i - \bar{x}}{s}$$

- List는 일반적인 값을 유연하게 저장하기 좋다.
- NumPy Array는 vector와 matrix의 numerical computation에 적합하다.
- NumPy에서는 반복문 없이 배열 전체에 연산을 적용할 수 있다.

Python Example: List vs. NumPy Array

```
scores_list = [82, 60, 91, 73, 96]
scores_array = np.array(scores_list)

print(scores_list)
print(scores_array)

print(scores_array + 5)
print(scores_array * 2)
```

NumPy Array에서는 산술 연산이 각 element에 한 번에 적용된다.

Python Example: Creating a DataFrame

```
# Import pandas with the conventional alias pd
import pandas as pd

# Each dictionary key becomes a column name
data = {
    "Name": ["A", "B", "C", "D"],
    "Major": ["AI", "Data", "AI", "Business"],
    "Score": [85, 92, 78, 88]
}

# Convert the dictionary to a pandas DataFrame
students = pd.DataFrame(data)

print(students)
print(students.dtypes)
```


Dictionary, Column, and Series

Dictionary에서 DataFrame으로 변환되는 흐름을 구분해 보자.

Dictionary	DataFrame	선택 결과
Key "Name"	column name "Name"	Series
Key "Major"	column name "Major"	Series
Key "Score"	column name "Score"	Series

- **Column:** table 안의 하나의 열이라는 구조적 개념
- **Series:** pandas에서 하나의 column을 선택했을 때 반환되는 1차원 object

`students["Score"]`은 Score column을 선택한다는 뜻이고, 그 결과의 object type은 Series이다.

Python Example: Checking DataFrame Structure

```
print(students.head())  
students.info()  
print(students.shape)  
print(students.columns)  
print(students.index)
```

- shape: (row count, column count)
- columns: column name collection
- index: row label collection

Index and ID Are Different

DataFrame의 **index**와 데이터 안의 **ID column**은 같은 것이 아니다.

Index	ID	Major	Final
0	1	AI	78
1	2	Data	85
2	3	AI	92

- Index는 row를 찾기 위한 label이다.
- ID는 dataset 안에 저장된 identifier이다.
- `iloc[0]`은 첫 번째 row를 의미하며 ID가 0이라는 뜻이 아니다.

Python Example: Selecting Columns

```
# Select one column: returns a Series
score = students["Score"]
print(score)
print(type(score))

# Select multiple columns: returns a DataFrame
selected = students[["Name", "Score"]]
print(selected)
print(type(selected))
```

하나의 column 선택은 Series, 여러 column 선택은 DataFrame을 반환한다.

Python Example: Student Dataset

```
students = pd.DataFrame({  
    "ID": [1, 2, 3, 4, 5, 6],  
    "Major": ["AI", "Data", "AI", "Business", "Data", "AI"],  
    "GradeLevel": [1, 2, 3, 2, 4, 1],  
    "StudyTime": [2.5, 3.0, 4.0, 1.5, 5.0, 2.0],  
    "Final": [78, 85, 92, 70, 95, 80]  
})  
  
print(students)
```

이후 filtering, sorting, grouping 실습에서는 이 DataFrame을 계속 사용한다.

Python Example: Boolean Filtering

```
# Create a Boolean condition for every row
condition = students["Final"] >= 80
print(condition)

# Keep rows where the condition is True
high_score = students[condition]
print(high_score)
```

`students["Final"] >= 80`이 True/False를 만들고, DataFrame이 True인 row만 선택한다.

Python Example: Multiple Conditions

```
selected = students[
    (students["StudyTime"] >= 3) &
    (students["Final"] >= 85)
]
print(selected)

need_attention = students[
    (students["StudyTime"] < 2) |
    (students["Final"] < 75)
]
print(need_attention)
```

pandas condition에서는 &가 AND, |가 OR 역할을 하며 각 조건은 괄호로 묶는다.

Python Example: Creating a New Column

```
# Add a Boolean column using a logical condition
students["NeedSupport"] = (
    (students["Final"] < 75) |
    (students["StudyTime"] < 2)
)

print(students[["ID", "StudyTime", "Final", "NeedSupport"]])
```

`students["NeedSupport"] = ...`는 기존 DataFrame에 NeedSupport라는 새로운 column을 추가한다.

Python Example: Sorting Data

```
print(students.sort_values("Final"))  
print(students.sort_values("Final", ascending=False))  
  
sorted_students = students.sort_values(  
    ["Major", "Final"],  
    ascending=[True, False]  
)  
print(sorted_students)
```

`sort_values()`는 지정한 column을 기준으로 row 순서를 재배열한다. `ascending=False`는 descending order이다.

Python Example: Group Summary

```
# Group rows by Major and summarize Final scores
major_mean = students.groupby("Major")["Final"].mean()
print(major_mean)

major_summary = students.groupby("Major")["Final"].agg(
    ["count", "mean", "std"]
)
print(major_summary)
```

`groupby("Major")`로 Major가 같은 row를 묶고, `["Final"]`을 선택한 뒤 `mean()`이나 `agg()`로 group별 summary를 계산한다.

Series Index after Grouping

다음 코드의 결과를 생각해 보자.

```
major_mean = students.groupby("Major")["Final"].mean()
```

결과가 Series라면:

- AI, Business, Data와 같은 Major category가 Series의 **index label**이 된다.
- 각 Major의 평균 Final score가 Series의 **values**가 된다.

DataFrame의 column name이 항상 Series의 index가 되는 것은 아니다. grouping이나 column-wise summary의 결과에 따라 index가 새롭게 정해질 수 있다.

Wide Data and Long Data

대표적인 두 형태가 있다.

Wide data: 반복 측정값을 여러 column에 저장

ID	Week1	Week2	Week3
1	70	75	80
2	65	72	78

Long data: 반복 측정값을 여러 row에 저장

ID	Week	Score
1	Week1	70
1	Week2	75
1	Week3	80

Long format은 visualization과 statistical modeling에서 편리한 경우가 많다.

Python Example: Wide to Long with melt()

```
# Create a wide-format DataFrame
wide = pd.DataFrame({
    "ID": [1, 2],
    "Week1": [70, 65],
    "Week2": [75, 72],
    "Week3": [80, 78]
})

# Convert repeated week columns into rows
long = wide.melt(
    id_vars="ID",
    var_name="Week",
    value_name="Score"
)

print(long)
```

Understanding melt()

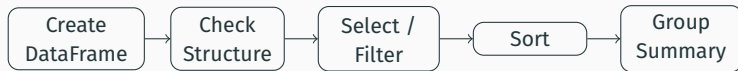
`melt()`의 주요 argument를 살펴보자.

- `id_vars="ID"`: 그대로 유지할 identifier column
- `var_name="Week"`: Week1, Week2, Week3라는 기존 column name을 저장할 새 column
- `value_name="Score"`: 실제 측정값을 저장할 새 column

Wide에서 여러 column이었던 Week1, Week2, Week3가 Long에서는 Week column의 category가 된다.

Coding Workflow for Table Data

Week 2에서 배운 coding 흐름을 연결하면 다음과 같다.



이번 주에는 복잡한 분석보다 table data를 읽고 조작하는 기본 동작을 정확히 익히는 것이 중요하다.

Practice Example 1: Data Type Classification

다음 variable의 type을 분류해 보자.

1. Student ID
2. Major
3. GradeLevel
4. StudyTime
5. Final
6. Satisfaction: low, medium, high

숫자로 저장되어 있는가보다 variable이 실제로 무엇을 의미하는가를 기준으로 판단한다.

Practice Example 2: Selection and Filtering

```
print(students[["ID", "Major", "Final"]])

high_final = students[students["Final"] >= 85]
print(high_final)

long_study = students[students["StudyTime"] >= 3]
print(long_study)
```

질문: Final 85점 이상인 학생과 StudyTime 3시간 이상인 학생은 각각 누구인가?

Practice Example 3: Sorting and Grouping

```
print(students.sort_values("Final", ascending=False))

print(students.groupby("Major")["Final"].mean())

print(
    students.groupby("Major")["StudyTime"]
    .agg(["count", "mean", "max"])
)
```

질문: 어떤 Major의 Final 평균이 가장 높은가? Major별 StudyTime 평균은 어떻게 다른가?

Practice Example 4: Derived Column

```
students["Performance"] = students["Final"] >= 85

print(students[["ID", "Major", "Final", "Performance"]])
print(students.groupby("Major")["Performance"].mean())
```

Boolean value의 평균은 True를 1, False를 0처럼 계산하므로 category별 비율을 간단히 확인할 수 있다.

Practice Example 5: Wide to Long

```
wide_scores = pd.DataFrame({
    "ID": [1, 2, 3],
    "Week1": [70, 75, 80],
    "Week2": [72, 78, 84],
    "Week3": [76, 80, 88]
})

long_scores = wide_scores.melt(
    id_vars="ID",
    var_name="Week",
    value_name="Score"
)
print(long_scores)
```

Checklist for Week 2

- Numerical, categorical, ordinal variable을 구분할 수 있는가?
- Discrete와 continuous variable의 차이를 설명할 수 있는가?
- Structured data와 unstructured data를 구분할 수 있는가?
- Row, column, observation, variable, value를 설명할 수 있는가?
- Dictionary의 Key와 DataFrame의 column name을 구분할 수 있는가?
- 하나의 DataFrame column을 선택하면 Series가 반환된다는 것을 이해하는가?
- Boolean filtering과 sorting을 수행할 수 있는가?
- `groupby()`의 동작을 설명할 수 있는가?
- Wide format을 Long format으로 변환할 수 있는가?

Review Exercise

1. student ID, major, grade level, study time, final score를 포함하는 작은 DataFrame을 만드시오.
2. 각 variable을 identifier, categorical, ordinal, numerical로 분류한다.
3. Final score를 descending order로 정렬하시오.
4. StudyTime이 3시간 이상인 학생만 filtering 하여.
5. Major별 Final 평균을 계산하시오.
6. Final이 75점 미만이거나 StudyTime이 2시간 미만인 학생을 NeedSupport로 표시하여라.
7. Week1-Week3 score를 가진 wide data를 만들고 long data로 변환하여라.
8. 각 결과를 코드만 제시하지 말고 각각 한 문장으로 해석하시오.

Review Exercise: 코드 정답 예시

```
# Create a small student dataset
students = pd.DataFrame({
    "ID": [1, 2, 3, 4, 5, 6],
    "Major": ["AI", "Data", "AI", "Business", "Data", "AI"],
    "GradeLevel": [1, 2, 3, 2, 4, 1],
    "StudyTime": [2.5, 3.0, 4.0, 1.5, 5.0, 2.0],
    "Final": [78, 85, 92, 70, 95, 80]
})

print(students)
```

Review Exercise: 코드 정답 예시

1. DataFrame 생성

- 하나의 row는 한 명의 학생이라는 **observation**을 나타낸다.
- ID, Major, GradeLevel, StudyTime, Final은 각각 하나의 **variable**이다.

2. Variable type 분류

Variable	Type
ID	Identifier
Major	Categorical
GradeLevel	Ordinal
StudyTime	Numerical
Final	Numerical

GradeLevel은 숫자로 저장되어 있지만 1학년, 2학년, 3학년, 4학년처럼 순서를 나타내므로 **Ordinal variable**로 해석할 수 있다.

Review Exercise: 코드 정답 예시

3. Final score 기준 descending order 정렬

```
sorted_students = students.sort_values(  
    "Final",  
    ascending=False  
)  
  
print(sorted_students)
```

Review Exercise: 코드 정답 예시

4. StudyTime이 3시간 이상인 학생 **filtering**

```
selected_students = students[
    students["StudyTime"] >= 3
]

print(selected_students)
```

sort_values()는 row의 순서를 바꾸는 작업이고, **Filtering**은 조건을 만족하는 row만 선택하는 작업이다.

Review Exercise: 코드 정답 예시

5. Major별 Final score 평균

```
major_mean = students.groupby("Major")["Final"].mean()

print(major_mean)
```

6. NeedSupport column 생성

```
students["NeedSupport"] = ((students["Final"] < 75) | (students["StudyTime"] < 2))

print( students[["ID", "StudyTime", "Final", "NeedSupport"]])
```

- `groupby("Major")`는 Major가 같은 학생들을 하나의 group으로 묶는다.
- `["Final"].mean()`은 각 Major group의 Final 평균을 계산한다.
- `|`는 두 조건 중 하나라도 만족하면 True가 되는 **OR** 연산이다.

Review Exercise: 코드 정답 예시

7. Wide data 생성

```
wide = pd.DataFrame({"ID": [1, 2, 3], "Week1": [70, 65, 80], "Week2": [75, 72, 85],  
                    "Week3": [80, 78, 90] })  
  
print(wide)
```

Wide data를 Long data로 변환

```
long = wide.melt( id_vars="ID", var_name="Week", value_name="Score" )  
  
print(long)
```

Wide data에서는 Week1-Week3가 각각 separate column으로 저장되어 있지만, Long data에서는 **Week**와 **Score**라는 variable로 변환되고 각 측정값이 별도의 row에 저장된다.

Review Exercise: Interpretation

8. 결과를 한 문장으로 해석하는 예

- **Sorting:** Final score를 내림차순으로 정렬한 결과, 가장 높은 점수를 받은 학생부터 순서대로 확인할 수 있다.
- **Filtering:** StudyTime이 3시간 이상인 학생만 선택하여 상대적으로 학습 시간이 긴 학생들을 확인할 수 있다.
- **Group Summary:** Major별 평균 Final score를 비교하여 전공별 성적 수준의 차이를 살펴볼 수 있다.
- **NeedSupport:** Final score가 75점 미만이거나 StudyTime이 2시간 미만인 학생을 추가적인 학습 지원이 필요한 대상으로 표시하였다.
- **Wide to Long:** 반복 측정된 Week1-Week3 score를 Long format으로 변환하면 Week와 Score를 각각 하나의 variable로 다룰 수 있다.

Python 코드를 실행하는 것에서 끝나지 않고, 어떤 **data**를 선택하거나 변환했으며 그 결과가 무엇을 의미하는지 함께 설명하는 것이 중요하다.

본 교육콘텐츠는 교육부와 한국연구재단의 지원을 받아 제작한 AI융합
인재양성사업단의 콘텐츠입니다.